Week 3 – Handson 6: Kafka Integration with C# (Detailed Report)

Objective

To understand Apache Kafka architecture and develop a simple chat application using C# where one application publishes messages and another consumes them in real time.

Software Requirements

- .NET 8 SDK
- Visual Studio Code/Visual Studio 2022
- Java JDK
- Apache Kafka
- Apache ZooKeeper
- Confluent.Kafka NuGet Package

Introduction

Apache Kafka is a distributed event streaming platform used to build high-performance, fault-tolerant applications. It enables communication between applications through events or messages. Kafka is widely used in banking, e-commerce, IoT, logging systems, and microservices.

Kafka Architecture

Kafka consists of Producers, Consumers, Topics, Partitions, Brokers, and ZooKeeper (or KRaft in newer versions). Producers publish messages to Topics. Topics are divided into Partitions for scalability. Brokers store and manage topics. Consumers subscribe to topics and receive messages independently.

Key Concepts

Producer: Sends messages.
Consumer: Reads messages.
Topic: Logical channel.
Partition: Improves scalability.
Broker: Kafka server.
ZooKeeper: Coordinates brokers in traditional Kafka deployments.

Folder Structure

```
WebApi_Handson6/
├── Program.cs
├── Producer.cs
├── Consumer.cs
├── Models/ChatMessage.cs
├── Properties/launchSettings.json
└── README.md
```

Program.cs

```
using Confluent.Kafka;
```

```
Console.WriteLine("Kafka Chat Application");
Console.WriteLine("1. Producer\n2. Consumer");
```

Producer.cs

```
using Confluent.Kafka;
public class Producer
{
 public async Task SendMessage(string message)
 {
  var config=new ProducerConfig{BootstrapServers="localhost:9092"};
  using var producer=new ProducerBuilder<Null,string>(config).Build();
  await producer.ProduceAsync("chat-topic",new Message<Null,string>{Value=message});
 }
}
```

Consumer.cs

```
using Confluent.Kafka;
public class Consumer
{
 public void Receive()
 {
  var config=new ConsumerConfig{BootstrapServers="localhost:9092",GroupId="chat-
group",AutoOffsetReset=AutoOffsetReset.Earliest};
  using var consumer=new ConsumerBuilder<Ignore,string>(config).Build();
  consumer.Subscribe("chat-topic");
  while(true){var result=consumer.Consume(); Console.WriteLine(result.Message.Value);}
 }
}
```

ChatMessage.cs

```
public class ChatMessage
{
 public string User{get;set;}="";
 public string Message{get;set;}="";
}
```

Kafka Installation

1. Install Java.
2. Download Apache Kafka.
3. Start ZooKeeper.
4. Start Kafka Server.
5. Create topic using kafka-topics.bat.
6. Run Producer.
7. Run Consumer.
8. Verify messages are exchanged.

Advantages

- High throughput
- Fault tolerant
- Scalable
- Durable message storage
- Real-time processing

Applications

Banking transaction processing, log aggregation, chat systems, IoT telemetry, fraud detection, recommendation systems and microservices.

Expected Output

Messages entered by the Producer are immediately displayed in the Consumer console, demonstrating successful event streaming.

Conclusion

This handson demonstrates the integration of Apache Kafka with C#. It explains Kafka architecture, producer-consumer communication, topic-based messaging, and real-time event streaming. Kafka is widely adopted in enterprise applications because of its scalability, reliability, and performance.